Online

Object Oriented Programming

Computer Systems Engineering

José Manuel Rodríguez Cantú

00612676

Activity 3: Class hierarchy design for a hospital software system

Module 3: Inheritance and Polymorphism

José Abdón Espínola González

Date:

July 24, 2026

1. Description of the solution

This activity models the people of a regional hospital with an object-oriented class hierarchy. The design rests on the three ideas the activity asks for: inheritance, abstraction and overloading. There are five files: the abstract class **Person**, its three subclasses **Doctor**, **Patient** and **Guard**, and a test program that creates one of each and registers them.

Person is an abstract superclass. It stores the two data fields every person shares, **name** and **age**, and exposes them through *getName* and *getAge*. It also declares an abstract method, *register*, with no body. Because the class is abstract it cannot be created on its own; it only serves as the common base for the specific people. That is abstraction: the shared idea of a person lives in one place and each subclass fills in the rest.

Doctor, **Patient** and **Guard** extend Person, so they inherit name, age and their getters. Each subclass adds only what makes it different: Doctor adds a department, Patient adds an illness, and Guard adds a shift and an optional phone. Every subclass constructor calls *super(name, age)*, so the shared data is set by the Person constructor instead of being repeated in each class.

Each subclass writes its own *register* method. When register is called, Java runs the version that belongs to the real object, so a Doctor prints "Welcome Doctor!", a Patient prints "Welcome Patient!" and a Guard prints "Welcome Guard!". This is the idea from the learning goal, different people performing the same activity in their own way, and in Java it is called polymorphism.

Guard shows overloading with two constructors: one receives a phone number and the other does not. The shorter constructor calls the longer one with *this(name, age, shift, null)*, so the setup code is written a single time and a guard can be created either way. The test uses John, created without a phone, and Kevin, created with one, to exercise both constructors.

2. Source code

`Person.java`

```java
// Abstract superclass shared by every person in the hospital system.
public abstract class Person {
    private String name; // the person's name
    private int age;     // the person's age in years

    // Sets the name and age common to every person.
    public Person(String nameP, int ageP) {
        this.name = nameP;
        this.age = ageP;
    }

    public String getName() { return name; }
    public int getAge() { return age; }

    // Each subclass greets its own kind of person, so this stays abstract.
    public abstract void register();
}
```

Doctor.java

```java
// A doctor is a Person who also belongs to a medical department.
public class Doctor extends Person {
    private String department; // the department the doctor works in

    // Creates a doctor with a name, an age and a department.
    public Doctor(String nameD, int ageD, String departmentD) {
        super(nameD, ageD); // reuse the Person constructor for the shared data
        this.department = departmentD;
    }

    public String getDepartment() { return department; }

    @Override
    public void register() {
        System.out.println("Welcome Doctor!");
    }
}
```

Patient.java

```java
// A patient is a Person who is admitted with a certain illness.
public class Patient extends Person {
    private String illness; // the illness the patient is treated for

    // Creates a patient with a name, an age and an illness.
    public Patient(String nameP, int ageP, String illnessP) {
        super(nameP, ageP); // reuse the Person constructor for the shared data
        this.illness = illnessP;
    }

    public String getIllness() { return illness; }

    @Override
    public void register() {
        System.out.println("Welcome Patient!");
    }
}
```

Guard.java

```java
// A guard is a Person who covers a shift and may have a contact phone.
// It shows constructor overloading: with or without a phone number.
public class Guard extends Person {
    private String shift; // the shift the guard covers (for example, Morning)
    private String phone; // the guard's contact phone (optional)

    // Guard without a phone: reuses the other constructor passing null.
    public Guard(String nameG, int ageG, String shiftG) {
        this(nameG, ageG, shiftG, null); // constructor chaining (overloading)
    }

    // Guard with a phone number.
    public Guard(String nameG, int ageG, String shiftG, String phoneG) {
        super(nameG, ageG); // reuse the Person constructor for the shared data
        this.shift = shiftG;
        this.phone = phoneG;
    }

    public String getShift() { return shift; }
    public String getPhone() { return phone; }

    @Override
    public void register() {
        System.out.println("Welcome Guard!");
    }
}
```

HospitalTest.java

```java
// Test program: creates the four requested people and registers each one.
public class HospitalTest {
    public static void main(String[] args) {
        // Test data requested in the activity
        Doctor doctor = new Doctor("Joseph", 41, "Neurologist");
        Patient patient = new Patient("Richard", 78, "Chronic Headache");
        Guard guardNoPhone = new Guard("John", 39, "Morning");                        //
3-arg constructor
        Guard guardWithPhone = new Guard("Kevin", 43, "Afternoon", "+52 232 456345"); //
4-arg constructor

        System.out.println("=== Hospital registration system ===");

        // Doctor
        System.out.println();
        System.out.println("Doctor");
        System.out.println("  Name: " + doctor.getName());
        System.out.println("  Age: " + doctor.getAge());
        System.out.println("  Department: " + doctor.getDepartment());
        doctor.register();

        // Patient
        System.out.println();
        System.out.println("Patient");
        System.out.println("  Name: " + patient.getName());
        System.out.println("  Age: " + patient.getAge());
        System.out.println("  Illness: " + patient.getIllness());
        patient.register();

        // Guard without a phone (shorter overloaded constructor)
        System.out.println();
        System.out.println("Guard");
        System.out.println("  Name: " + guardNoPhone.getName());
        System.out.println("  Age: " + guardNoPhone.getAge());
        System.out.println("  Shift: " + guardNoPhone.getShift());
        System.out.println("  Phone: " + (guardNoPhone.getPhone() == null ? "not provided"
: guardNoPhone.getPhone()));
        guardNoPhone.register();

        // Guard with a phone (longer overloaded constructor)
        System.out.println();
        System.out.println("Guard");
        System.out.println("  Name: " + guardWithPhone.getName());
        System.out.println("  Age: " + guardWithPhone.getAge());
        System.out.println("  Shift: " + guardWithPhone.getShift());
        System.out.println("  Phone: " + (guardWithPhone.getPhone() == null ? "not
provided" : guardWithPhone.getPhone()));
        guardWithPhone.register();
    }
}
```

3. Execution evidence

The five files compile with `javac` without errors or warnings, and the test program prints the data of the four people together with the welcome message produced by each `register` method. John, built with the three-argument constructor, shows "not provided" as phone, while Kevin, built with the four-argument constructor, shows his number. The screenshot shows the compilation and the run:

```
$ javac Person.java Doctor.java Patient.java Guard.java HospitalTest.java
$ java HospitalTest
=== Hospital registration system ===

Doctor
  Name: Joseph
  Age: 41
  Department: Neurologist
Welcome Doctor!

Patient
  Name: Richard
  Age: 78
  Illness: Chronic Headache
Welcome Patient!

Guard
  Name: John
  Age: 39
  Shift: Morning
  Phone: not provided
Welcome Guard!

Guard
  Name: Kevin
  Age: 43
  Shift: Afternoon
  Phone: +52 232 456345
Welcome Guard!
```

Each person is registered through its own `register` method, so the same call produces a different greeting depending on the object. The two Guard objects confirm that both overloaded constructors work: the one without a phone falls back to "not provided" and the one with a phone keeps the number.